Multi-Agent AI Recruitment Assessment System

DOCUMENT 1: Complete Python Backend Architecture & Source Code Explanation

Executive Overview

This document provides a comprehensive technical walkthrough of every Python file in the backend repository. The backend is engineered as an asynchronous, modular micro-framework powered by FastAPI, LangGraph multi-agent workflows, Pydantic data schemas, SQLAlchemy ORM with SQLite, Whisper speech-to-text models, and isolated subprocess code execution sandboxing.

Component Layer	Directory / Files	Primary Functionality
Core Server	main.py, core/config.py	FastAPI application setup, CORS middleware, lifespan initialization, route binding.
Data & Persistence	models/candidate.py, database/	SQLAlchemy relational schemas, DB session lifecycle, and initial mock data seeding.
Validation Schemas	schemas/*.py	Pydantic v2 strict schema models for resumes, assessments, fair scoring, and HR dossiers.
Specialized AI Agents	agents/*.py (6 Agents)	Autonomous agents: Parsing, Matching, Tech Execution, Screening, Fairness, HR Approval.
Core Services	services/*.py	LiteLLM / Gemini gateway, Docker sandbox execution, Vector search embeddings, Whisper STT.
Orchestration Workflows	workflows/recruitment_graph.py	LangGraph StateGraph workflow engine with conditional branching and state preservation.
REST API Endpoints	api/routes/*.py (9 Routes)	FastAPI routes for candidates, resumes, coding, screening, scoring, shortlisting, and WebSockets.

1. Core Server Layer

backend/app/main.py

Initializes the FastAPI application instance. Configures CORS middleware to allow cross-origin requests from the Vite frontend (port 5173). Manages database startup lifecycle via init_db() and seed_database(). Includes the global API routers and starts WebSocket broadcaster.

backend/app/core/config.py

Centralizes environment variables and application settings using pydantic_settings.BaseSettings. Manages API keys for Google Gemini and OpenAI, database connection strings, default LLM model selections, and sandbox execution timeouts.

2. Database & Data Models Layer

backend/app/models/candidate.py

Defines SQLAlchemy ORM relational models:
• **CandidateModel**: Stores candidate metadata, parsed resume JSON, raw text, stage statuses (Pending, Running, Completed), and individual assessment scores.
• **JobDescriptionModel**: Stores job listings, company names (Stripe, Anthropic, Scale AI, Snowflake), required skills, preferred skills, and experience levels.
• **AssessmentRecordModel**: Audit trail recording every agent execution timestamp, model used, latency in ms, and full JSON payload.
• **HumanReviewModel**: Stores human recruiter shortlisting/rejection decisions, notes, and timestamps.

backend/app/database/session.py & seed_data.py

Configures SQLite engine with thread-safe session factories. seed_data.py populates 6 enterprise company job listings and 3 diverse benchmark candidates for zero-configuration testing.

3. Autonomous Specialized AI Agents

Agent 1: Parsing Agent (parsing_agent.py)

Parses unstructured resume text into a standardized Pydantic schema (skills, experience chronology, education, projects, certifications) while enforcing PII masking rules.

Agent 2: Matching Agent (matching_agent.py)

Computes semantic cosine similarity between candidate competencies and job descriptions, identifying Strong Matches, Partial Matches, and Missing Skill Gaps.

Agent 3: Tech Agent (tech_agent.py)

Coordinates technical coding evaluation, running candidate solutions against hidden unit tests in an isolated sandbox, calculating runtime latency, cyclomatic complexity, and code quality score.

Agent 4: Screening Agent (screening_agent.py)

Deconstructs transcribed interview responses using the STAR methodology (Situation, Task, Action, Result), evaluates communication clarity, filler words, and protocol compliance.

Agent 5: Fairness Agent (fairness_agent.py)

Enforces strict demographic-blind scoring. Strips candidate names, gender, race, age, and school prestige to generate an objective weighted Merit Score out of 100.

Agent 6: HR Shortlisting Agent (hr_agent.py)

Consolidates all agent evaluation vectors into an executive candidate dossier and drafts automated personalized notification emails upon human approval.

4. Core Backend Services

LLM Gateway (llm_gateway.py)

Unified gateway integrating Google Generative AI (gemini-3.6-flash) and OpenAI with automated JSON schema repair, rate limiting, and demo fallback simulator.

Code Execution Engine (code_execution.py)

Subprocess-hardened execution sandbox. Enforces 5-second timeout limits, memory isolation, and captures stdout/stderr against parameterized test cases.

Vector Search & Embeddings (embeddings.py)

Dense text embedding generator and cosine similarity calculator for semantic skill and keyword alignment.

Transcription Service (transcription.py)

Speech-to-Text engine supporting Whisper acoustic models for converting candidate audio recordings into timestamped text transcripts.

5. LangGraph Workflow & API Layer

backend/app/workflows/recruitment_graph.py

Builds the LangGraph StateGraph pipeline. Defines state transitions across ResumeParser -> SkillMatcher -> TechAssessment -> AudioScreening -> FairScoring -> HRApproval with cyclic retry loops and conditional branching.

backend/app/api/routes/

- candidates.py: CRUD operations, job listings, and /portal/upload-resume-file binary PDF/Word parsing endpoint.
- resume.py: Resume parsing and Pydantic JSON schema validator.
- matching.py: Skill match and gap analysis triggers.
- coding.py: Problem catalog and sandbox code execution.
- screening.py: Whisper audio transcription and STAR analysis endpoint.
- scoring.py: Demographic-blind scoring matrix computation.
- hr.py: Human-in-the-loop shortlist/reject approval and email dispatch.
- workflow.py & ws.py: Full pipeline execution trigger and real-time WebSocket broadcast.

6. Automated Unit & Integration Tests

backend/tests/test_agents.py & root conftest.py

Contains 6 automated integration tests validating 100% of backend agents and the complete LangGraph pipeline. Executable directly with python -m pytest backend/tests/test_agents.py -v.